Milestone 4: RISC-V Vector Assembly Implementation

Kalman Filter Project

May 16, 2026

1 Introduction

This report details the implementation, verification, and performance analysis of the Linear Kalman Filter (LKF) and Extended Kalman Filter (EKF) vectorised using the RISC-V Vector (RVV) extension.

2 Function-by-Function Walkthrough

The vectorised implementations leverage the RISC-V RVV 1.0 extension to accelerate the core matrix operations. The vector length is configured dynamically using `vsetvli`.

2.1 Matrix Multiplication (`mat_mul_vec`)

The matrix multiplication kernel computes $C = A \times B$. The loop order is chosen to write the output matrix C exactly once per output chunk to minimise memory traffic.

```
# For k = 0..K-1:
fld     ft0, 0(a6)                # scalar A[i][k]
vle64.v  v4, (a7)                 # B[k][j..j+VL-1]
vfmacc.vf v0, ft0, v4            # v0 += A[i][k] * B[k][j..j+VL-1]
```

2.2 Element-wise Operations

Addition, subtraction, and scaled addition treat the matrices as flat arrays and use `vfadd.vv`, `vfsub.vv`, and `vfmacc.vf`.

```
vle64.v  v0, (a1)       # load A chunk
vle64.v  v4, (a2)       # load B chunk
vfadd.vv v0, v0, v4    # v0 = A + B
vse64.v  v0, (a0)       # store to C
```

2.3 Matrix-Vector Multiplication (`mat_vec_mul_vec`)

This function uses an ordered reduction (`vfredosum.vs`) for deterministic and reproducible summation across runs.

```
vfmul.vv v8, v0, v4              # element-wise product
vfredosum.vs v12, v8, v12         # v12[0] = sum(v8)
```

2.4 Matrix Transposition (`mat_transpose_vec`)

Transposition is vectorised by column using strided loads (`vlse64.v`) and unit-stride stores (`vse64.v`).

```
vlse64.v v0, (a6), s4             # strided load, stride = N*8
vse64.v  v0, (a7)                  # unit-stride store
```

3 Numerical Verification

The vectorised output is compared against the scalar assembly implementation to ensure numerical accuracy ($\epsilon \leq 10^{-9}$).

3.1 Average Absolute Error per Joint

3.2 Average Absolute Error per State Component

3.3 Overall Error Bounds

- LKF Max Error: 4.22e-15
- LKF Min Error: 0.00e+00
- EKF Max Error: 6.66e-15
- EKF Min Error: 0.00e+00

All errors are well within the required 10^{-9} tolerance.

3.4 Direct Comparison: Scalar vs Vector

4 Performance Analysis

The speedup analysis compares the scalar (M3) code against the vectorised (M4) code.

4.1 Instruction Count Reduction

The instruction count was measured using the QEMU plugin `libinsn.so`.

Joint	LKF Vector Error	EKF Vector Error
pelvis	2.45e-16	6.73e-18
L5	1.19e-16	8.91e-17
L3	5.14e-17	1.36e-16
T12	9.27e-17	8.88e-17
T8	1.79e-17	1.53e-16
neck	2.26e-16	1.92e-16
head	8.05e-17	5.75e-17
shoulderRight	9.65e-17	1.29e-16
upperArmRight	2.04e-16	8.24e-17
forearmRight	3.49e-16	8.12e-17
handRight	1.47e-16	4.50e-16
shoulderLeft	1.45e-16	2.61e-16
upperArmLeft	5.07e-17	3.36e-16
forearmLeft	9.28e-17	9.28e-17
handLeft	8.21e-17	2.15e-16
upperLegRight	1.36e-16	2.88e-16
lowerLegRight	7.12e-17	6.06e-18
footRight	2.51e-17	1.16e-17
toeRight	1.20e-16	1.86e-17
upperLegLeft	2.52e-16	1.55e-16
lowerLegLeft	1.59e-16	1.81e-16
footLeft	8.70e-17	3.60e-16
toeLeft	4.06e-17	9.09e-17

Table 1: Average absolute error per joint for vectorised LKF and EKF.

4.2 Speedup Measurement

The wall-clock time was measured over multiple frames using the `perf_compare` utility running in QEMU. Note that under QEMU emulation, vector operations are simulated using scalar loops, resulting in a wall-clock slowdown, however the instruction count correctly reflects the reduction in architectural instructions executed.

Component	LKF Vector Error	EKF Vector Error
px	1.54e-16	1.78e-16
vx	5.50e-17	3.15e-16
ax	3.41e-18	2.07e-17
jx	1.34e-19	8.56e-19
py	3.09e-16	4.33e-16
vy	4.25e-16	7.26e-16
ay	2.57e-17	4.52e-17
jy	1.00e-18	1.80e-18
pz	1.17e-17	4.07e-17
vz	4.89e-16	5.26e-17
az	3.25e-17	3.55e-18
jz	1.20e-18	1.36e-19

Table 2: Average absolute error per component for vectorised LKF and EKF.

Joint/Component	M3 Scalar Error	M4 Vector Error
LKF pelvis	2.33e-16	2.45e-16
LKF px	1.54e-16	1.54e-16
EKF head	7.18e-17	5.75e-17
EKF vy	5.03e-16	7.26e-16

Table 3: Comparison of errors showing vectorisation did not degrade numerical accuracy.

Filter	M3 Scalar Insns	M4 Vector Insns	Reduction Ratio
LKF	6,665,955,583	3,547,738,380	1.88x
EKF	6,668,931,253	3,550,632,658	1.88x

Table 4: Instruction count reduction from M3 scalar to M4 vector.

Filter	M3 Scalar Time (s)	M4 Vector Time (s)	Speedup (QEMU emulated)
LKF	25.84	116.04	0.22x
EKF	24.55	106.91	0.23x

Table 5: Execution time speedup comparison.